

# Tema 4.6: Framework Front-End

## Avanzado (Angular 19, Signals, Guards, Interceptores JWT y Testing)

---

### Resumen

---

Este tema profundiza en la ingeniería de aplicaciones web de página única (*Single-Page Applications* - SPAs) de nivel empresarial utilizando el framework **Angular 19**. Tomando como base la introducción conceptual a componentes e inyección de dependencias (`inject()`), esta sesión avanzada explora la arquitectura orientada a **Componentes Standalone**, el nuevo modelo de reactividad con **Angular Signals** (`signal()`, `computed()`, `effect()`), el control de flujo moderno (`@if`, `@for`, `@switch`), el consumo asíncrono con **RxJS** y `HttpClient`, la seguridad cliente mediante **HTTP Interceptors** (`HttpInterceptorFn`) para tokens JWT, la protección de rutas con **Route Guards** (`CanActivateFn`), la gestión de Formularios Reactivos (*Reactive Forms*) y el desarrollo de **Pruebas Unitarias** con `TestBed` y `provideHttpClientTesting()`. Incluye un ejercicio de codificación en vivo (*Live Coding*) paso a paso integrado directamente en la lección.

---

### Concepto Principal (El 'Qué')

---

#### 4.6.1 Arquitectura Standalone y Reactividad con Angular Signals

A partir de Angular 14+ y consolidado en Angular 19, el framework ha sustituido los tradicionales `NgModule` por la **Arquitectura Standalone**. Cada componente, directiva o pipe declara sus dependencias directamente mediante la propiedad `imports` de su decorador `@Component`.

#### El Nuevo Paradigma: Angular Signals

Las **Signals** introducen un motor de reactividad granular que notifica al framework exactamente qué nodos del DOM deben actualizarse cuando cambia un valor, eliminando la necesidad de re-evaluar todo el árbol de componentes mediante la detección de cambios masiva (*Zone.js ZoneCoalescing*).

```

import { Component, signal, computed } from '@angular/core';

@Component({
  selector: 'app-contador',
  standalone: true,
  template: `
    <p>Cantidad: {{ cantidad() }}</p>
    <p>Total a Pagar: {{ total() | currency:'USD' }}</p>
    <button (click)="incrementar()">+1 Paquete</button>
  `
})
export class ContadorComponent {
  // 1. Writable Signal (Estado Mutable)
  cantidad = signal<number>(1);
  precioUnitario = signal<number>(25.0);

  // 2. Computed Signal (Estado Derivado de Solo Lectura)
  total = computed(() => this.cantidad() * this.precioUnitario());

  incrementar(): void {
    this.cantidad.update(v => v + 1);
  }
}

```

## 4.6.2 Control de Flujo Moderno en Plantillas

Angular 17+ reemplazó las directivas estructurales legadas (\*ngIf, \*ngFor, \*ngSwitch) por bloques sintácticos nativos integrados directamente en el compilador del framework:

```

<!-- Bloque Condicional @if / @else -->
@if (cargando()) {
  <div class="spinner">Cargando paquetes desde el servidor...</div>
} @else if (error()) {
  <div class="alert-danger">{{ error() }}</div>
} @else {
  <!-- Bloque de Iteración @for con tracking obligatorio -->
  <ul class="lista-envios">
    @for (paquete of paquetes(); track paquete.id) {
      <li class="item-paquete">
        <span>{{ paquete.codigoRastreo }}</span>
        <!-- Bloque de Selección @switch -->
        @switch (paquete.estado) {

```

```

        @case ('REGISTRADO') { <span class="badge-blue">Registrado</span> }
        @case ('EN_TRANSITO') { <span class="badge-orange">En Transito</span> }
        @case ('ENTREGADO') { <span class="badge-green">Entregado</span> }
        @default { <span class="badge-gray">Desconocido</span> }
    }
</li>
} @empty {
    <p>No se encontraron paquetes registrados en el sistema.</p>
}
</ul>
}

```

### 4.6.3 Interceptores HTTP y Protección de Rutas con JWT

En arquitecturas empresariales desacopladas, el cliente SPA debe adjuntar automáticamente el token de autenticación **JWT (RFC 7519)** en el encabezado HTTP `Authorization: Bearer <token>` de cada petición dirigida a la API RESTful.

#### 1. Interceptor HTTP Funcional (`HttpInterceptorFn`)

```

import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { AuthService } from '../services/auth.service';

export const jwtInterceptor: HttpInterceptorFn = (req, next) => {
    const authService = inject(AuthService);
    const token = authService.obtenerToken();

    if (token) {
        const reqClonada = req.clone({
            setHeaders: {
                Authorization: `Bearer ${token}`
            }
        });
    };
    return next(reqClonada);
}

return next(req);
};

```

#### 2. Guard de Enrutamiento Funcional (`CanActivateFn`)

```
import { CanActivateFn, Router } from '@angular/router';
import { inject } from '@angular/core';
import { AuthService } from '../services/auth.service';

export const authGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const router = inject(Router);

  if (authService.estaAutenticado()) {
    return true;
  }

  router.navigate(['/login'], { queryParams: { returnUrl: state.url } });
  return false;
};
```

---

#### 4.6.4 Pruebas Unitarias de Componentes y Servicios

Angular incluye integración nativa con **Jasmine** y **TestBed** para validar de forma aislada la lógica de servicios y componentes.

```
import { TestBed } from '@angular/core/testing';
import { provideHttpClientTesting, HttpTestingController }
  from '@angular/common/http/testing';
import { provideHttpClient } from '@angular/common/http';
import { PaqueteService } from './paquete.service';

describe('PaqueteService', () => {
  let service: PaqueteService;
  let httpMock: HttpTestingController;

  beforeEach(() => {
    TestBed.configureTestingModule({
      providers: [
        PaqueteService,
        provideHttpClient(),
        provideHttpClientTesting()
      ]
    });
    service = TestBed.inject(PaqueteService);
    httpMock = TestBed.inject(HttpTestingController);
  });
```

```

afterEach(() => {
  httpMock.verify(); // Verifica que no queden peticiones pendientes
});

it('debe obtener la lista de paquetes via GET', () => {
  const paquetesDummy = [
    { id: 1, codigoRastreo: 'PKG-01', pesoKg: 10.5, estado: 'REGISTRADO' }
  ];

  service.obtenerPaquetes().subscribe(paquetes => {
    expect(paquetes.length).toBe(1);
    expect(paquetes[0].codigoRastreo).toBe('PKG-01');
  });

  const req = httpMock.expectOne('http://localhost:8080/api/v1/paquetes');
  expect(req.request.method).toBe('GET');
  req.flush(paquetesDummy); // Simula la respuesta JSON del servidor
});
});

```

---

## Analogía (La Intuición)

---

Imagine la operación de un **Aeropuerto Internacional Inteligente**:

- **El Componente Standalone es una Terminal Autónoma:** Cada terminal del aeropuerto cuenta con sus propias puertas de abordaje, baños y servicios de equipaje sin depender de una central rígida (`NgModule`). Si necesita conectar un tren de transbordo, simplemente importa ese servicio específico.
- **Signals son las Pantallas de Salidas y Llegadas:** Cuando un vuelo cambia de puerta o sufre un retraso, no se envía una alarma por altavoz a todo el aeropuerto para revisar cada pasajero. La pantalla específica del vuelo (*Signal*) parpadea y actualiza únicamente a los pasajeros que están frente a esa puerta.
- **El Interceptor HTTP es el Punto de Control de Pasaportes:** Cada vez que un pasajero (petición HTTP) intenta salir del país en cualquier vuelo hacia el servidor back-end, el agente de aduana (*HttpInterceptorFn*) revisa su pasaporte, le coloca el sello oficial (*Header Authorization: Bearer JWT*) y lo deja abordar.

- **El Route Guard es el Guardia de Seguridad de la Sala VIP:** Antes de permitirle a un pasajero entrar a la zona de descanso exclusiva (*Ruta Protegida*), el guardia (*CanActivateFn*) verifica si su boleto incluye acceso premium. Si no cumple las condiciones, lo reorienta cordialmente a la sala de espera pública (*/login*).

---

## Ejercicio de Codificación en Vivo paso a paso (Live Coding)

---

En este ejercicio práctico construiremos en vivo la **Arquitectura Front-End Angular 19 de LogiExpress**, integrando Signals, Interceptores JWT, Guards y Servicios REST.

### Paso 1: Configuración Global de Proveedores (`app.config.ts`)

```
import { ApplicationConfig, provideZoneChangeDetection }
  from '@angular/core';
import { provideRouter, withComponentInputBinding }
  from '@angular/router';
import { provideHttpClient, withInterceptors }
  from '@angular/common/http';
import { routes } from './app.routes';
import { jwtInterceptor } from './interceptors/jwt.interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes, withComponentInputBinding()),
    provideHttpClient(withInterceptors([jwtInterceptor]))
  ]
};
```

---

### Paso 2: Modelo e Interfaces de Dominio (`domain/paquete.model.ts`)

```
export type EstadoPaquete = 'REGISTRADO' | 'EN_TRANSITO' | 'ENTREGADO';

export interface Paquete {
  id?: number;
  codigoRastreo: string;
```

```
descripcion: string;
pesoKg: number;
estado: EstadoPaquete;
clienteId: number;
}
```

---

### Paso 3: Servicio de Autenticación con Signals (`services/auth.service.ts`)

```
import { Injectable, signal, computed } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class AuthService {
  // Signal privado mutable con la clave JWT
  private tokenSignal = signal<string | null>(
    localStorage.getItem('jwt_token')
  );

  // Signal computado que deriva el estado de autenticación
  estaAutenticado = computed(() => this.tokenSignal() !== null);

  guardarToken(token: string): void {
    localStorage.setItem('jwt_token', token);
    this.tokenSignal.set(token);
  }

  obtenerToken(): string | null {
    return this.tokenSignal();
  }

  cerrarSesion(): void {
    localStorage.removeItem('jwt_token');
    this.tokenSignal.set(null);
  }
}
```

---

### Paso 4: Servicio de Paquetes con RxJS y HttpClient (`services/paquete.service.ts`)

```

import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { Paquete } from '../domain/paquete.model';

@Injectable({
  providedIn: 'root'
})
export class PaqueteService {
  private readonly http = inject(HttpClient);
  private readonly apiUrl = 'http://localhost:8080/api/v1/paquetes';

  obtenerPaquetes(): Observable<Paquete[]> {
    return this.http.get<Paquete[]>(this.apiUrl);
  }

  registrarPaquete(paquete: Paquete): Observable<Paquete> {
    return this.http.post<Paquete>(this.apiUrl, paquete);
  }
}

```

---

## Paso 5: Componente Standalone con Reactividad y Signals

(`components/paquete-list.component.ts`)

```

import { Component, inject, OnInit, signal } from '@angular/core';
import { CommonModule } from '@angular/common';
import { PaqueteService } from '../services/paquete.service';
import { Paquete } from '../domain/paquete.model';

@Component({
  selector: 'app-paquete-list',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './paquete-list.component.html',
  styleUrls: ['./paquete-list.component.css']
})
export class PaqueteListComponent implements OnInit {
  private readonly paqueteService = inject(PaqueteService);

  // Signals para gestionar el estado visual del componente
  paquetes = signal<Paquete[]>([]);
  cargando = signal<boolean>(true);
}

```

```

    errorMsg = signal<string | null>(null);

    ngOnInit(): void {
        this.cargarPaquetes();
    }

    cargarPaquetes(): void {
        this.cargando.set(true);
        this.paqueteService.obtenerPaquetes().subscribe({
            next: (datos) => {
                this.paquetes.set(datos);
                this.cargando.set(false);
            },
            error: (err) => {
                this.errorMsg.set('Falla al conectar con la API de LogiExpress');
                this.cargando.set(false);
            }
        });
    }
}

```

---

## Paso 6: Plantilla del Componente con Control de Flujo Moderno

(paquete-list.component.html)

```

<section class="container">
    <h2>Gestion de Paquetes LogiExpress</h2>

    @if (cargando()) {
        <div class="loading-spinner">Cargando datos asincronos...</div>
    } @else if (errorMsg()) {
        <div class="alert-error">{{ errorMsg() }}</div>
    } @else {
        <table class="data-table">
            <thead>
                <tr>
                    <th>ID</th>
                    <th>Codigo Rastreo</th>
                    <th>Peso (kg)</th>
                    <th>Estado</th>
                </tr>
            </thead>
            <tbody>

```

```

    @for (item of paquetes(); track item.id) {
      <tr>
        <td>{{ item.id }}</td>
        <td><strong>{{ item.codigoRastreo }}</strong></td>
        <td>{{ item.pesoKg }} kg</td>
        <td>
          @switch (item.estado) {
            @case ('REGISTRADO') { <span class="badge blue">Registrado</span> }
            @case ('EN_TRANSITO') { <span class="badge orange">En Transito</span> }
            @case ('ENTREGADO') { <span class="badge green">Entregado</span> }
          }
        </td>
      </tr>
    } @empty {
      <tr>
        <td colspan="4">No existen paquetes registrados.</td>
      </tr>
    }
  </tbody>
</table>
}
</section>

```

---

## Paso 7: Configuración de Rutas con Lazy Loading (`app.routes.ts`)

```

import { Routes } from '@angular/router';
import { authGuard } from '../guards/auth.guard';

export const routes: Routes = [
  {
    path: 'login',
    loadComponent: () => import('../components/login.component')
      .then(m => m.LoginComponent)
  },
  {
    path: 'paquetes',
    loadComponent: () => import('../components/paquete-list.component')
      .then(m => m.PaqueteListComponent),
    canActivate: [authGuard]
  },
  {
    path: '',

```

```
    redirectTo: 'paquetes',  
    pathMatch: 'full'  
  }  
];
```

---

## Trampas Comunes

---

### Fuga de Memoria por Suscripciones Infinitas a Observables:

- *Error:* Suscribirse manualmente a Observables en componentes (`.subscribe()`) sin cancelar la suscripción al destruir el componente (`ngOnDestroy`).
- *Solución:* Utilizar **Angular Signals** (`toSignal()`) o el operador `takeUntilDestroyed()` de RxJS para limpiar automáticamente los flujos asíncronos.

### Olvidar Registrar el Interceptor en `app.config.ts`:

- *Error:* Crear la función `HttpInterceptorFn` pero no incluir `provideHttpClient(withInterceptors([jwtInterceptor]))` en el archivo de configuración global, provocando que las peticiones salgan sin el encabezado `Authorization`.
- *Solución:* Verificar que todos los interceptores funcionales estén declarados dentro de `withInterceptors([])`.

### Modificar Signals Mutables Directamente sin Mutadores (`set/update`):

- *Error:* Intentar modificar una Signal mediante asignaciones de variables directas como `this.miSignal = 10`.
- *Solución:* Usar la API oficial de Signals: `.set(nuevoValor)` para reemplazar el valor o `.update(fn)` para modificar el estado existente.

## Pregunta de Reflexión

---

En la arquitectura de aplicaciones cliente Angular a gran escala:

¿Qué ventajas operativas y de rendimiento ofrece migrar de un modelo de reactividad basado en Detección de Cambios masiva (*Zone.js*) hacia un esquema puramente basado en **Angular Signals** en conjunto con la estrategia `ChangeDetectionStrategy.OnPush`?